# Notas Unificadas
# Testing de Software

**Ingeniería y Calidad de Software · 06/10/2026**

*Integración de apuntes de clase y materiales compartidos*

**Idea central: el testing busca hacer visibles defectos cuya presencia se presume. Encontrar defectos es señal de un testing exitoso, pero las pruebas no pueden asegurar por sí solas la calidad ni demostrar ausencia total de defectos.**

## 1. Testing, calidad y aseguramiento de calidad

El testing o prueba de software es una actividad orientada a encontrar defectos. Se intenta poner el software en situaciones que permitan observar comportamientos incorrectos. Una prueba exitosa es aquella que logra encontrar defectos; que una ejecución no encuentre ninguno no significa que el producto esté libre de ellos.

- El testing no garantiza la calidad del producto.
- No puede garantizar que no existan errores o defectos; solo permite hacer visibles los que se manifiestan en las situaciones probadas.
- El testing llega relativamente tarde si se espera a disponer de una versión ejecutable; por eso conviene trabajar también en prevención y detección temprana.

### Calidad

La calidad es subjetiva y se entiende como el nivel de adecuación de un producto o servicio a las necesidades y expectativas del cliente o usuario.

- Necesidades: vinculadas principalmente con los requerimientos funcionales.
- Expectativas: suelen relacionarse con requerimientos no funcionales y muchas veces no son manifiestas; el usuario puede asumirlas como obvias y utilizarlas para evaluar el producto.

### QA y testing

El aseguramiento de calidad comprende actividades que acompañan el ciclo de vida del software con una mirada preventiva. El testing forma parte de ese esfuerzo, pero no equivale a todo el QA. La prevención suele ser menos costosa que detectar y corregir problemas tardíamente.

## 2. Validación y verificación (V&V)

| Concepto | Pregunta de clase | Foco |
|---|---|---|
| Validación | ¿Construimos el producto correcto? | Adecuación del producto a la necesidad real del usuario. |

| | | |
|---|---|---|
| Verificación | ¿Construimos el producto correctamente? | Funcionamiento/construcción correcta respecto de lo especificado. |

**Para poder validar y verificar se necesita partir de requerimientos. Los requerimientos son también una fuente fundamental para diseñar pruebas.**

## 3. Error, defecto y falla

La distinción trabajada en clase depende del momento en que se introduce y se detecta el problema.

- Error: se introduce y se detecta dentro de la misma fase o etapa en la que se produjo.
- Defecto: atraviesa la fase donde se introdujo y se detecta posteriormente, en otra etapa. Cuanto más tiempo permanece, mayor es su "edad" y, en general, mayor puede ser el costo de corregirlo.
- Falla: mal funcionamiento observable del sistema que ocurre como consecuencia de un defecto. No todos los defectos necesariamente se manifiestan como fallas.

### Edad del defecto

Es el tiempo transcurrido desde que el defecto fue introducido hasta que se detecta. Detectarlo tempranamente evita que otras decisiones o componentes se construyan sobre él.

## 4. Caja negra y caja blanca

- Caja negra: prueba dinámica. Se ejecuta el software y se observan entradas, salidas y comportamiento sin basarse en el código interno.
- Caja blanca: se observa/analiza el código; en los apuntes se la contrapone con la prueba dinámica de caja negra.

## 5. Clasificación de defectos: severidad y prioridad

Son dos dimensiones diferentes. La severidad indica cuánto afecta técnicamente el defecto al producto; la prioridad expresa la urgencia del negocio para resolverlo.

### Severidad

| Nivel | Interpretación trabajada en clase |
|---|---|
| 1. Bloqueante / invalidante | No se puede utilizar el sistema o continuar probando. |
| 2. Crítico | No se puede cumplir el objetivo de una funcionalidad y no existe alternativa viable. |
| 3. Mayor | La funcionalidad principal falla, pero existe alguna alternativa o "plan B". |
| 4. Menor | No impide completar el objetivo, aunque el comportamiento no es el correcto. |
| 5. Cosmético | Problemas de presentación, ortografía, apariencia o usabilidad que no impiden la tarea principal. |

La severidad la determina una mirada técnica, normalmente el equipo de testing.

### Prioridad

- Urgente: no puede esperar para resolverse.
- Alta.
- Media.
- Baja.

La prioridad depende del contexto del negocio y del tiempo que puede esperarse para corregir el problema. Severidad y prioridad son independientes: un defecto grave puede no ser el primero a corregir y uno menor puede requerir atención inmediata según la entrega o el negocio.

## 6. Niveles de prueba

| Nivel | Quién / dónde | Foco |
|---|---|---|
| Unitarias | Desarrollador · workflow de implementación | Unidad pequeña de código. Relacionadas con TDD; son la excepción práctica a la independencia entre quien desarrolla y quien prueba. |
| Integración / interfaces | Workflow de implementación; automatizables con CI | Interacción entre componentes e interfaces. El resultado integrado puede constituir un build. |
| Sistema / versión | Equipo de testing · workflow de prueba | Versión integrada del incremento o producto, evaluada como un todo. |
| Aceptación de usuario | Usuario final · despliegue | El usuario decide si acepta o no el producto; el equipo acompaña el proceso. |

## 7. Ambientes de trabajo y prueba

Un ambiente es la configuración del espacio de trabajo, repositorio y herramientas necesarias para desarrollar o probar. Los ambientes de desarrollo y testing deben ser distintos para evitar interferencias y favorecer la objetividad y reproducibilidad.

- Desarrollo: implementación y corrección.
- Testing / prueba: recibe una versión identificada para que el equipo de testing trabaje con independencia y datos controlados.
- Preproducción: ambiente de despliegue semejante a producción; se utiliza para aceptación y pruebas sensibles a infraestructura, como performance, estrés o seguridad.
- Producción: entorno real con usuarios y datos reales.

**“En mi máquina andaba” resume uno de los problemas que se intenta evitar separando ambientes. Aun así, diferencias de hardware o infraestructura pueden hacer que preproducción y producción no se comporten exactamente igual.**

## 8. Caso de prueba

El caso de prueba es uno de los entregables centrales del testing: un conjunto preciso de instrucciones, datos y resultados esperados que permite alcanzar un objetivo de prueba de manera sistemática y reproducible.

- Debe permitir que otra persona reproduzca la prueba y llegue al mismo resultado.
- Es más específico que un escenario general: define condiciones, pasos, datos y respuesta esperada.
- Los casos pueden derivarse de requerimientos e historias de usuario, incluyendo sus criterios de aceptación.
- También pueden apoyarse en información de relevamiento, especificaciones y otros documentos disponibles del producto.

### Problema de las pruebas exhaustivas

No es viable probar todas las entradas y combinaciones posibles. El testing tiene costo y tiempo limitados, por lo que se busca diseñar la menor cantidad razonable de casos sin perder efectividad para detectar defectos. La selección debe cerrar en términos de costo/beneficio y riesgo.

### Métodos de caja negra mencionados

- Partición de equivalencias: agrupa valores que deberían comportarse de forma semejante y elige representantes.
- Análisis de valores límite: concentra las pruebas en los bordes de las clases, donde suelen aparecer errores de rango o comparación.

## 9. Proceso de testing y artefactos

**Planificación de pruebas:** Define objetivos, alcance, riesgos, estrategia, niveles/tipos de prueba, ambiente, recursos y criterios. Artefacto: Plan de pruebas.

**Diseño de casos de prueba:** Deriva y prioriza casos; prepara pasos, datos de prueba y estados iniciales. Artefactos: casos de prueba y datos de prueba.

**Ejecución:** Entra una versión identificada del software junto con la información preparada en el diseño; se ejecutan los casos y se comparan resultados.

**Reporte y seguimiento:** Se documentan defectos con la información necesaria para reproducirlos. El reporte retroalimenta al desarrollo, que corrige y entrega una nueva versión.

**Planificación y diseño pueden adelantarse usando requerimientos, sin esperar a que exista una versión ejecutable. Esto evita que todo el trabajo de prueba empiece recién al final.**

## 10. Ciclos de prueba y regresión

Un ciclo de prueba comprende la ejecución del conjunto de casos previsto sobre una misma versión y el reporte de resultados y defectos. Desarrollo corrige y entrega una nueva versión, que inicia un nuevo ciclo.

1. Ciclo 0: ejecutar pruebas y reportar defectos.
2. Desarrollo corrige y entrega una nueva versión.
3. Ciclo 1: volver a probar. Si quedan defectos o aparecen nuevos, se repite el proceso.

En clase se planteó como situación ideal disponer de al menos dos ciclos: uno para detectar defectos y otro para comprobar las correcciones. En la práctica pueden requerirse más.

### Regresión

Hacer regresión significa volver a ejecutar, sobre la nueva versión, los casos ya definidos para comprobar no solo las correcciones puntuales sino también que lo que antes funcionaba continúe funcionando. Si se prueban únicamente los defectos reportados, pueden quedar roturas ocultas en otras funciones.

## 11. Ideas clave para estudiar

- Testing exitoso = encontrar defectos; no equivale a demostrar ausencia de defectos.
- Calidad y testing no son sinónimos: el testing es una actividad dentro del aseguramiento de calidad.
- Validar pregunta por el producto correcto; verificar, por construirlo correctamente.
- Error, defecto y falla se distinguen por dónde se detecta el problema y por su manifestación observable.
- Severidad ≠ prioridad.
- Los niveles siguen la integración creciente: unitario → integración → sistema → aceptación.
- Los ambientes deben separarse para controlar versiones, datos e interferencias.
- El caso de prueba debe ser específico y reproducible.
- No existen pruebas exhaustivas viables: hay que seleccionar casos con criterio.
- El testing se prepara antes de ejecutar: planificación → diseño → ejecución → reporte.
- La regresión vuelve a comprobar el comportamiento previo después de una corrección.

*Notas unificadas elaboradas únicamente a partir de los materiales de clase compartidos para el 06/10/2026.*